Card Run Blackjack 20× — Math Pack for Certification

Oct 16, 2026 · @Bob

Card Run Blackjack 20× returns **94.368%** at best play on the board, and its DOUBLE OR NOTHING hand now returns **100.07% to 100.09%**. Even a player who uses every button optimally gets about **97.49%** over a whole round, which is still under 100%. This is the math pack for the certification lab; every figure is exact or checked against a simulation of the shipped game.

Summary

Build `card-run-blackjack-classic-launch.html`, sha256 `381595b2ec0103398f92886e7429afc9624c3a53d6e68ebcd7e86fde01f2708c`, 131,176 bytes, run bar v170. The only change from the previous build: in the DOUBLE OR NOTHING hand the dealer now stands on all 17s (it hit soft 17). The base game already stood on all 17s, so one dealer rule now covers the whole game.

Item	Value
Base game RTP, best play (all cards on the table)	**94.368%**
Base game RTP, rules-sheet basic play	93.448%
Base game RTP, worst play (lowest documented)	**81.950%**
Top pay	20× the bet, 1 in 209,049 rounds
MAX WIN	20,000× the bet (an operator may lower it, never raise it)
DOUBLE OR NOTHING, best play	**100.0910%**
DOUBLE OR NOTHING, basic strategy	**100.0721%**
Whole round, cash out at the first chance	94.3679%
Whole round, double to the 1,000× cap	95.2299%

Item	Value
Whole round, best possible use of the bar	**about 97.49%**, upper bound 97.489%
Bets	$1.00 to $100.00 in $0.10 steps
Rounding	Whole cents, rounded down; none occurs on the base game or any double

Two things the lab should know.

1. **The double game is no longer capped at 100%.** The shared rule "each double returns at most 100%" no longer holds for Blackjack. Declare the double as 100.07% (basic strategy) to 100.09% (best play).
2. **The change request's worst case of 94.4% is too low.** It covers only a player who doubles every time. A player who also uses DOUBLE HALF can reach **97.49%**, found exactly by dynamic programming (see Round-level RTP). The claim that every round strategy stays under 100% still holds, with 2.5 points to spare.

Rules as built

The player is dealt four hands against one dealer, draws cards from an infinite deck, and is paid for how many hands win, how many make 21 and how many are blackjacks.

- **Deck.** Every card is an independent draw from 13 ranks; ten-valued cards are 4 in 13.
- **Dealer.** One card up, draws after the four hands, hits 16 or less, **stands on all 17s**.
- **Player choices.** Hit or stand on hard 12–16 and soft 17. All other totals play automatically: hard 11 or less and soft 16 or less hit; hard 17 or more and soft 18 or more stand.
- **Ties win.** A player blackjack against a dealer blackjack ties, and the BJ column pays. A dealer blackjack beats every other hand.
- **After any win** the run bar opens: CASH OUT, DOUBLE OR NOTHING or DOUBLE HALF (see Run phase).

The DOUBLE OR NOTHING hand. One player hand against the dealer, same deck, **dealer stands on all 17s**. The player must hit hard 11 or less and soft 16 or less, chooses on hard 12–16 and soft 17–18, and stands on anything higher. Ties on 19 and 20 win; any other tie is a push and the hand is dealt again. A two-card blackjack beats a 21 made with three or more cards. A win pays exactly 2× the stake, a loss pays 0.

Pays and board math

The three pay columns add. All pays are multiples of the bet.

Column	1	2	3	4
WINS (hands won)	0	0.3×	1.2×	2×
MADE 21 (hands that make 21)	0.4×	1×	2×	4×
BJ (blackjacks)	0.5×	2×	10×	18×

The WINS column starts at two hands won (0.3×). The top pay is 20×: four blackjacks plus four wins (1 in 209,049 rounds). Four blackjacks against a dealer blackjack pay 18× (1 in 4,207,107). The "20×" in the name is a target, not a cap.

The exact return is computed by backward induction over every player decision and a forward pass over every outcome (`exact_board.js`). At best play the columns return WINS 74.179%, MADE 21 9.149% and BJ 11.039%, **94.368%** in total. Every board pay is a whole number of cents at every legal bet, so there is no rounding effect.

Pay distribution, best play (one board).

Pay (× bet)	Probability	1 in
20.0	0.0000047836	209,049
18.0	0.00000023769	4,207,107
12.4	0.000026390	37,893
12.0	0.00017403	5,746
11.2	0.00018466	5,415
10.4	0.0000015012	666,148
10.0	0.000017633	56,711
6.0	0.000018980	52,688
5.0	0.000059181	16,897
4.5	0.000057938	17,260
4.4	0.00071711	1,394

Pay (× bet)	Probability	1 in
4.0	0.0034633	289
3.6	0.00078748	1,270
3.5	0.00099214	1,008
3.2	0.0041494	241
3.0	0.0047850	209
2.9	0.0075999	132
2.7	0.0010908	917
2.5	0.030174	33
2.4	0.031066	32
2.3	0.0033395	299
2.2	0.0062144	161
2.1	0.0098390	102
2.0	0.14454	7
1.7	0.032681	31
1.6	0.039496	25
1.5	0.00013181	7,587
1.3	0.0047994	208
1.2	0.11300	9
1.0	0.0011143	897
0.9	0.0015421	648
0.8	0.031466	32
0.7	0.035658	28
0.5	0.039662	25
0.4	0.040279	25
0.3	0.10616	9

Pay (× bet)	Probability	1 in
0	0.30471	3

The table is the exact probability of each pay to five significant digits; `par.txt` has them in full precision. The hit rate at best play is 69.53% and the standard deviation is 0.9421× the bet.

Skill and strategy

Blackjack is a skill game: best play is 12.4 points above worst play on the board, and 0.92 points above the rules-sheet basic play. The displayed RTP is for best play, which uses every card already on the table.

Play	RTP	SD (× bet)	Hit rate
Best play (all cards on the table)	94.368%	0.9421	69.53%
Basic play (rules sheet)	93.448%	0.9246	72.48%
Standard blackjack chart	93.651%	0.9362	71.44%
Always hit at choices	89.842%	0.8859	74.98%
Random 50/50 at choices	87.516%	0.9118	69.04%
Always stand at choices	87.455%	0.9796	61.54%
Worst play	81.950%	0.8978	69.14%

What the rules sheet discloses. It lists best, basic and worst play, describes best play as using every card on the table, and spells out the basic play: always hit hard 12; hit hard 13 against 2 and 3; stand on hard 13–16 against 4–6 and on 14–16 against 2 and 3; hit 13–16 against 7 or higher; hit soft 17 except against 7.

The exact best-play decision table the simulations use is `out/optmasks.bin`, produced by `exact_board.js`. The lowest documented return anywhere is worst play at 81.95%, above the 75% (GLI-19 §4.7) and 80% (Missouri 11 CSR 45-5.190) minimums.

Run phase: DOUBLE OR NOTHING and the run bar

With the dealer standing on all 17s, DOUBLE OR NOTHING wins 50.0455% of resolved hands and returns 100.0910% at best play. The previous rule (dealer hits soft 17) gave 99.9077%.

The double hand, exact

Let V be the win chance of one resolved hand (pushes are dealt again). V solves V = W + P × V, where W and P are the win and push chances of a single deal; `rungame.js` solves it exactly, re-optimising the player's hit/stand choices against V. The return is 2V.

Play	RTP	Win chance	Push per deal	Deals per hand	RTP, dealer hits soft 17 (before)
Best play	**100.0910%**	50.0455%	4.052%	1.04223	99.9077%
Basic strategy	**100.0721%**	50.0360%	4.000%	1.04166	99.8981%
Worst play	92.0888%	46.0444%	3.683%	1.03824	91.9174%
Always stand	94.7814%	47.3907%	3.145%	1.03247	94.8811%
Always hit	97.3050%	48.6525%	4.541%	1.04757	96.8500%

- Best play: V = 0.50045487064506489, so 2V = 1.000910.
- **"Basic strategy" is the conventional no-doubling blackjack chart**: hard 12 stand against 4–6, else hit; hard 13–16 stand against 2–6, else hit; soft 17 hit; soft 18 stand against 2–8, hit against 9, 10 and ace. It reproduces the change request's before and after figures to four decimals.
- The rules sheet's own "basic play" is a board strategy with no rule for soft 18. Applied to this hand with soft 18 standing it returns 99.9579%; it is shown for information only.
- The edge per double is +0.091% at best play and +0.072% at basic strategy. **Declare 100.07% to 100.09%.**
- Standing on soft 17 is worth 0.183 points at best play and 0.174 at basic strategy.

The bar

After any win the bar offers **CASH OUT**, **DOUBLE OR NOTHING** and **DOUBLE HALF**, with a **RUN BANK** escrow.

- CASH OUT pays the RUN BANK plus the held amount.

- DOUBLE OR NOTHING plays the double with all of the held amount.
- DOUBLE HALF stakes half the held amount (rounded down to the cent) and puts the rest, with any odd cent, in the RUN BANK.
- A won double makes the held amount 2× the stake; the RUN BANK stays. A lost double returns the RUN BANK as the held amount (the banked-bust rule); with an empty RUN BANK the round ends at 0.
- After every result, in cents: clip the held amount to MAX WIN minus RUN BANK; end at MAX WIN; end at the **1,000× run cap** (1,000 × the first win; the double that crosses it pays in full); otherwise show the bar.
- DOUBLE needs a held amount of at least $0.02 and RUN BANK + 2 × held within MAX WIN. DOUBLE HALF needs half of at least $0.02 (held of $0.04 or more) and RUN BANK + the kept half + 2 × half within MAX WIN.
- Every double is an exact whole-cent doubling: no rounding at small amounts. RUN IT AGAIN and RUN HALF are not in this build.

The shipped bar code equals the shared module `run-bar.js` v170: 1,594,400 states compared, 0 differences.

Round-level RTP: the worst case is 97.49%, not 94.4%

The best a player can do with the bar is about 97.49% of the bet over a whole round. That is 3.1 points above cashing out and 2.3 above doubling every time, and still under 100%.

Why the double changes the round maths

Each double now wins with probability 0.500455, a +0.091% bet. The bar is therefore a slightly favourable betting game, limited by the 1,000× cap and MAX WIN. Before this change every double returned under 100% and cashing out was the best policy. Now every policy that doubles beats cash out, and the question is how much the best one can win.

Named policies (best play on the board and in the double)

The amount at stake is the round's own first win. The values are exact and the same at $1, $10 and $100.

Policy	Rule	RTP now	RTP before
P0	Cash out at the first bar	94.3679%	94.3679%
P2	DOUBLE OR NOTHING once, then cash out	94.4538%	94.2809%
P3	DOUBLE OR NOTHING while offered, to the cap	95.2299%	93.5010%
P4	A random button at every bar	94.4825%	94.2520%
P5	DOUBLE HALF once, then cash out	94.4108%	94.3244%

Closed forms, each matched to the bar model to 1e-12: P2 = P0 + (2V − 1) × E[first win doubled]; P5 = P0 + (2V − 1) × E[half]; P3 = E[first win × (2V)^10], because ten doublings reach 1,024×, past the 1,000× cap (94.368% × 1.000910^10 = 95.230%). P4 has no closed form: a full recursion over every state converges to 94.48248 (cut at path probability 1e-12, cut mass 0.0188%).

The exact optimum over every possible button sequence

`bar_optimum.cpp` values every state (held amount, RUN BANK) in whole cents by value iteration, to a relative residual of 1e-13, with the build's own offers, cap, minimum and banked-bust rule. Its offers match `run-bar.js` v170 on 1,351,200 states with 0 differences, and an independent evaluator confirms the values to 1e-12. For a first win of A cents, λ(A) is the best expected final amount divided by A.

First win A (cents)	States	λ(A)	Always DOUBLE	Always DOUBLE HALF
5	241,575	1.031066	1.009135	1.012794
10	567,875	1.032151	1.009135	1.021982
20	1,319,328	1.032639	1.009135	1.026800
40	3,033,454	1.032859	1.009135	1.029350
100	8,993,268	1.032996	1.009135	1.031040
200	20,282,073	1.033015	1.009135	1.031597
300	32,536,195	1.033024	1.009135	1.031792

Sizes 30, 50, 60, 70, 80, 90, 120, 130, 150, 160 and 170 are also solved (`par.txt` section 7). In every one the best first button is DOUBLE HALF.

- **The optimal player never cashes out before the cap.** At the start, and whenever the held amount is more than about 45% of everything at stake, it plays DOUBLE HALF. When the held amount is under about 25% of the total it mostly plays DOUBLE.
- **The limit:** λ(∞) = 1.03304 ± 0.00004, from Richardson extrapolation and least-squares fits. λ(A) is not strictly increasing (whole-cent halves add noise of up to 3.6e-5), so the bound uses $\hat{\lambda}$ = 1.03308.
- At basic-strategy play (V = 0.50036) the optimum is 1.0261 at A = 200 cents, and always DOUBLE gives 1.007232.
- **MAX WIN** only removes offers, so it can only lower the value; in the tightest case (MAX WIN equal to the run cap) λ falls by 1.6e-3.

Round-level worst case

Bet	Cash out	Always DOUBLE	Best bar play, estimate	Upper bound
$1	94.3679%	95.2299%	97.482%	97.489%
$10	94.3679%	95.2299%	97.487%	97.489%
$100	94.3679%	95.2299%	97.487%	97.489%

The estimate uses the solved value where one exists and a fitted λ(A) = 1.033055 − 0.0067 / A elsewhere. The upper bound is the board's 94.368% times $\hat{\lambda}$ = 1.03308 and assumes λ(A) ⩽ $\hat{\lambda}$ at every A, which the solved sizes support but do not prove. For any board policy the expected final amount is at most $\hat{\lambda}$ times the expected first win, and the first win is largest at best play, so changing the board play cannot raise the figure.

The change request's 94.4% is the always-double case on a 93.5% board (93.5% × 1.0009^10). On the shipped board's 94.368% the always-double figure is 95.23%, and the true worst case is 97.49%. Wherever the worst case is quoted, replace 94.4% with 97.49%.

Rounding, limits and RNG

Nothing in Blackjack is rounded: board pays are exact at every legal bet and every double is an exact 2×.

- **Rounding.** All pays are whole cents, rounded down. Every board pay is a multiple of 0.1× and every bet a multiple of 10 cents, so board pays are exact. DOUBLE HALF stakes the floor of half the cents and banks the rest.
- **Bets.** $1.00 to $100.00 in $0.10 steps.
- **MAX WIN.** 20,000× the bet; an operator may set it lower, never higher.
- **Run cap.** 1,000× the first win; the double that crosses it pays in full, so a run can pay more than 1,000×.
- **Minimums.** $0.02 for DOUBLE OR NOTHING, $0.04 for DOUBLE HALF.
- **RNG.** Every card comes from `crypto.getRandomValues` with rejection sampling; no outcome uses `Math.random`, and the game throws if `crypto` is missing. Production replaces the browser source with the platform RNG (open item).

Simulation evidence

Every exact value sits inside its simulation's 95% interval, except one chance miss that an independent follow-up cleared. All simulations run the shipped game scripts in Node with a seeded generator behind `crypto.getRandomValues`.

Base game and bar policies

Run	Rounds	Result
Base, best play, $10	2,000,000	94.344% [94.213%, 94.475%]; exact 94.368% inside
P0 cash out, $10	200,000	94.302% [93.890%, 94.714%]
P2 double once, $10	200,000	95.019% [94.302%, 95.735%]
P4 random button, $10	600,000	94.069% [93.291%, 94.848%]; exact 94.482% inside
P5 DOUBLE HALF once, $10	200,000	94.213% [93.708%, 94.718%]

P3 (double to the cap) is too heavy-tailed to test by simulation (hit rate 0.07%, SD 42.8× the bet), so it rests on its closed form. In total 37 checks were made; 36 are inside. The one miss is the P2 hit rate at $100: exact 34.796%, sample interval [34.825%, 35.417%], 2.15 standard errors; about 1.9 misses are expected by chance in 37 checks. A follow-up on 4

new seeds (400,000 rounds, chosen once, run once) gives 34.660% [34.513%, 34.807%], inside. Follow-ups are reported apart and never pooled. P4 follow-ups at $1, $10 and $100 are also inside.

The shipped code follows the exact bar model on every transition: 2,184,075 transitions over 2,200,000 simulated P4 rounds, 0 mismatches. A Monte Carlo of the bar model itself (25–50 million rounds) is inside at all three bets.

DOUBLE OR NOTHING through the shipped code

One resolved hand per call of the build's own `runGame`.

Play	Hands	RTP [95% CI]	Exact	Pushes per deal	Exact
Best play	10,000,000	100.135% [100.073%, 100.197%]	100.0910%	4.050% [4.038%, 4.062%]	4.052%
Basic strategy	10,000,000	100.099% [100.037%, 100.161%]	100.0721%	3.998% [3.986%, 4.010%]	4.000%

All four exact values are inside. A resolved hand pays 0 or 2×, so its SD is 1.00 and 10 million hands fix the RTP only to ±0.06 points while the edge is 0.09. These runs show consistency with the exact value; the exact value itself comes from the evaluator.

The optimal bar policy

- Bar-only model at A = 40 cents, 2 × 100 million rounds with the optimal policy: 1.03718 and 1.03355 × A, both containing the exact 1.032859 and neither containing always-DOUBLE (1.009135). Pooled, 9.7 standard errors above always-DOUBLE.
- Shipped code at A = 40 cents, 2 × 500,000 rounds: 0.971 and 1.048 × A, both containing the exact value. The SD is 37–41 × A, so this cannot tell the optimum from always-DOUBLE; it checks the code path only.

Same seeds as the previous build

11 of 44 chunks are identical round for round (all 8 base chunks and all 3 P0 chunks: sums, hit rates, maximum pays, RNG draws, full payout histograms). The other 33 (P2, P3, P4, P5) changed, as they must, because only the double's dealer changed. The exact board table is identical to the previous build's and the best-play decision table is byte-identical.

Browser checks of the rule change

Headless Chromium, 14 checks, all pass: a rigged dealer soft 17 (A, 6) stands with no third card; a rigged hard 16 (10, 6) hits; the felt reads "DEALER STANDS ON 17"; the double panel reads "WIN PAYS 2X · RETURNS 100.1%"; the rules sheet has the new dealer sentence and the "DOUBLE OR NOTHING, best play 100.1%" row; a round through DOUBLE OR NOTHING, DOUBLE HALF and CASH OUT pays correctly. The rules sheet shows RTPs to one decimal, so it reads 100.1%; this document and `par.txt` carry 100.09% and 100.07%.

Reproduce, files and open items

The pack folder `blackjack/` reproduces every number here. Each exact solver reads its constants and rule fragments from the build text and stops if they differ, and every output records the build's sha256. `README.txt` lists every command and every seed.

File	What it does
`consts.js`	Parses pays, limits and rule fragments from the build and asserts them, including that the old hit-soft-17 rule is gone
`exact_board.js`	Exact board maths: RTP by play, pay distribution, best-play table
`rungame.js`	Exact DOUBLE OR NOTHING hand under both dealer rules
`exact_runs.js`	Exact policy values P0, P2–P5 at $1, $10, $100
`bar_optimum.cpp`, `bar_optimum_run.js`	Exact optimum over every bar policy, checked against run-bar v170
`bar_policy_eval.js`, `bar_policy_sim.js`	Independent check of the optimum; the optimal policy through the shipped code
`round_worst.js`	Round-level worst case and extrapolation
`policy.js`, `harness.js`, `seedcrypto.js`, `strategy.js`	Run the shipped scripts with a seeded RNG and a bot
`mc_model.js`, `trace_check.js`, `trace_replay.sh`, `runbar_diff.js`	Model Monte Carlo and equality proofs between the shipped bar and the model

File	What it does
`double_sim.js`, `browser_smoke.js`	The double hand through the shipped code; browser rig tests
`aggregate.js`, `compare_prev.js`, `cr_report.js`	CI checks, same-seed comparison, change-request report
`par.txt`, `results.json`, `BLACKJACK-MATH.md`	The PAR sheet, every number as data, and this document in markdown

Open items for the lab and for the team.

1. **Declare the double hand as a range**, 100.07% to 100.09%, and the round worst case as 97.49% (see Round-level RTP). Withdraw the statement that every bar action returns under 100% for Blackjack.
2. **The optimum is solved exactly for first wins up to 300 cents.** Larger first wins (all of them at $100, most at $10) use a fitted curve, and the upper bound assumes $\lambda(A) \leqslant 1.03308$ for every A. Solving A = 3,000 cents would need roughly 300 million states; say if the lab wants it.
3. **"Basic strategy" for the double** is the conventional chart, as in the change request. The rules sheet's own basic play gives 99.9579% on this hand, and the sheet does not show a basic figure for the double.
4. **The rules sheet shows 100.1%** (one decimal, as for every RTP on the sheet). The change request asks for 100.09% and 100.07% in the rules text; say if the lab wants two decimals.
5. **Clover and Vault** appear in the change request's rules table; they are not in this build.
6. **RNG:** production replaces the browser source with the platform RNG and server-side outcome determination.